

git-drs

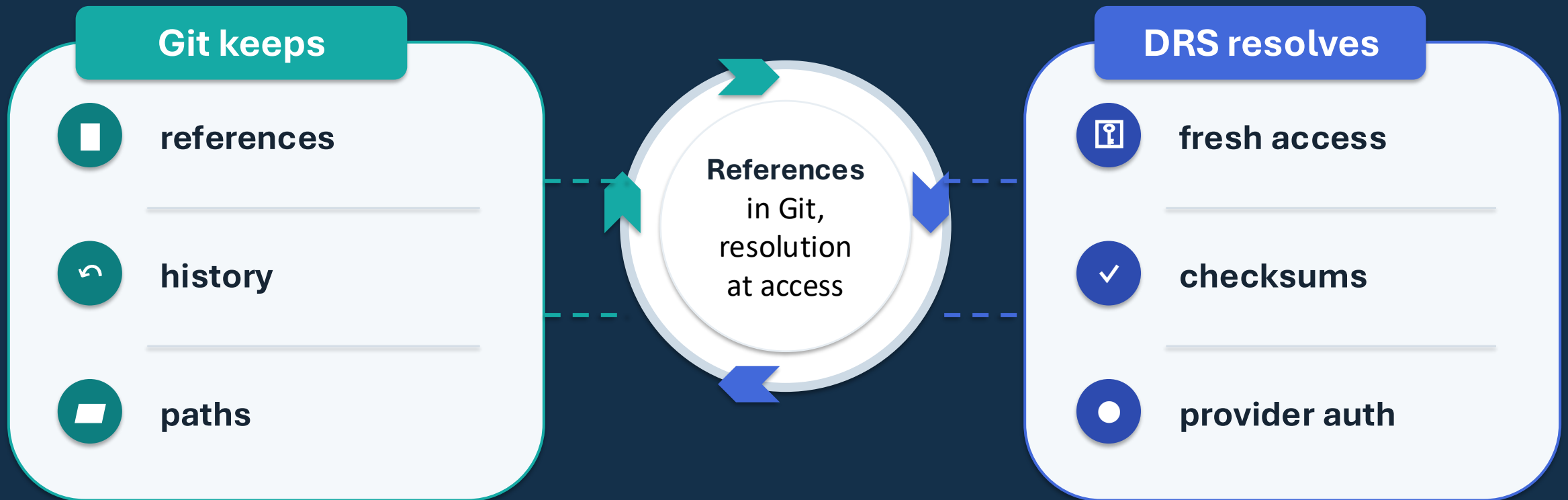
PUBLISHER: ADD THE DATA. VERSION THE REFERENCES. SHARE THE DATASET.

CONSUMER: CLONE THE REFERENCES. HYDRATE THE DATA. REPRODUCE THE RESULTS.



walsbr@ohsu.edu
<https://ellrottlab.org/>
Sept 2026

AnVIL + Terra meets git-drs



Git versions stable references; DRS supplies authorized, verified access to the bytes.

Clone a lightweight repository that versions stable DRS references to your dataset.
Authenticate to the data provider and hydrate the files you need—on demand.



Secure by Design

Authorization and tokens are resolved when data is accessed—never stored in Git.



Reproducible

Git versions the dataset structure and references so results can be reliably reproduced.



Collaborative

Share the same dataset layout with your team without duplicating data.



On-Demand Access

Hydrate only the files you need, when you need them.



Storage Agnostic

Works with any DRS-enabled data repository or cloud backend.

The git-drs Data Consumer Workflow

1

Clone Repository

Get the dataset references (not the data).

```
gcloud auth application-default login

git clone https://github.com/org/ANVIL_1000G_PRIMED_data_model.git
```



Your repository contains DRS pointers, not data.

2

Configure Terra Remote

Set up local access to AnVIL data.

```
git drs remote add anvil terra
--checkout hydrate
```



Authentication via Google ADC

3

Hydrate Files

Download and validate the files you are authorized to use.

```
git drs pull -I "*.tsv"

# or
git drs pull -I "sample.tsv"
```



DRS resolves, downloads, verifies, and hydrates.



Why git-drs?



Scales to any size

Reference large datasets without moving them.



Always up to date

Fresh metadata and access at the time of use.



Integrity guaranteed

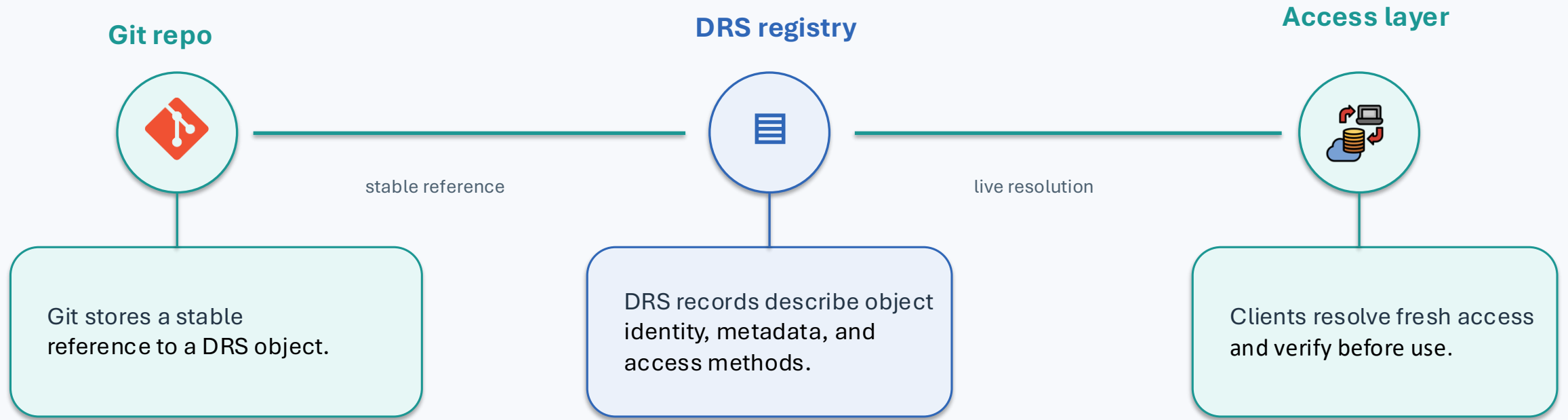
Size and SHA256 verified before use.



Fits your workflow

Git for versioning; git-drs for access.

DRS: object metadata + live access



Reference schema work: GA4GH DRS schemas and the writable DRS upload prototype branch.

Git stores a stable reference to a DRS object, not the bytes.

DRS records describe object identity, metadata, and live access methods.

Clients resolve a fresh access URL at read time and validate size/checksum before hydration.

Add local or existing data once, and git-drs records stable DRS references, checksums, and dataset structure. Then use ordinary Git to commit, version, and share the dataset definition—while the bytes remain in the right data repository.



Safe & Responsible

Data stays in approved repositories with proper access controls.



Versioned & Reproducible

Git versions the dataset structure and references, so every release can be reproduced.



Shareable

Share a lightweight repo, not the data. Collaborators hydrate with their own access.



Repository Agnostic

Works with any DRS-enabled repository or cloud backend.



Integrity by Design

References include size and SHA256. Consumers verify before using.

The git-drs Data Publisher Workflow

1

Add Data

Add local files or existing DRS objects to the repository.

```
git drs add data/  
# or add existing DRS URIs  
git drs add-ref drs://drs.anv...  
dataset.tsv
```



Files become DRS references with size and SHA256.

2

Commit & Version

Git versions the references and dataset structure.

```
git add .  
git commit -m "Add 1000G dataset"  
git tag v1.0.0
```



Every commit captures exactly which data, and which version.

3

Push & Share

Push to your remote and share like any other Git repository.

```
git push origin main  
git push origin v1.0.0
```



Collaborators clone the repo and hydrate with their own access.



Why publish with git-drs?



Lightweight to share

Share a small repo, not terabytes.



Built for data scale

DRS handles secure, scalable data access—Git handles everything else.



Future-proof

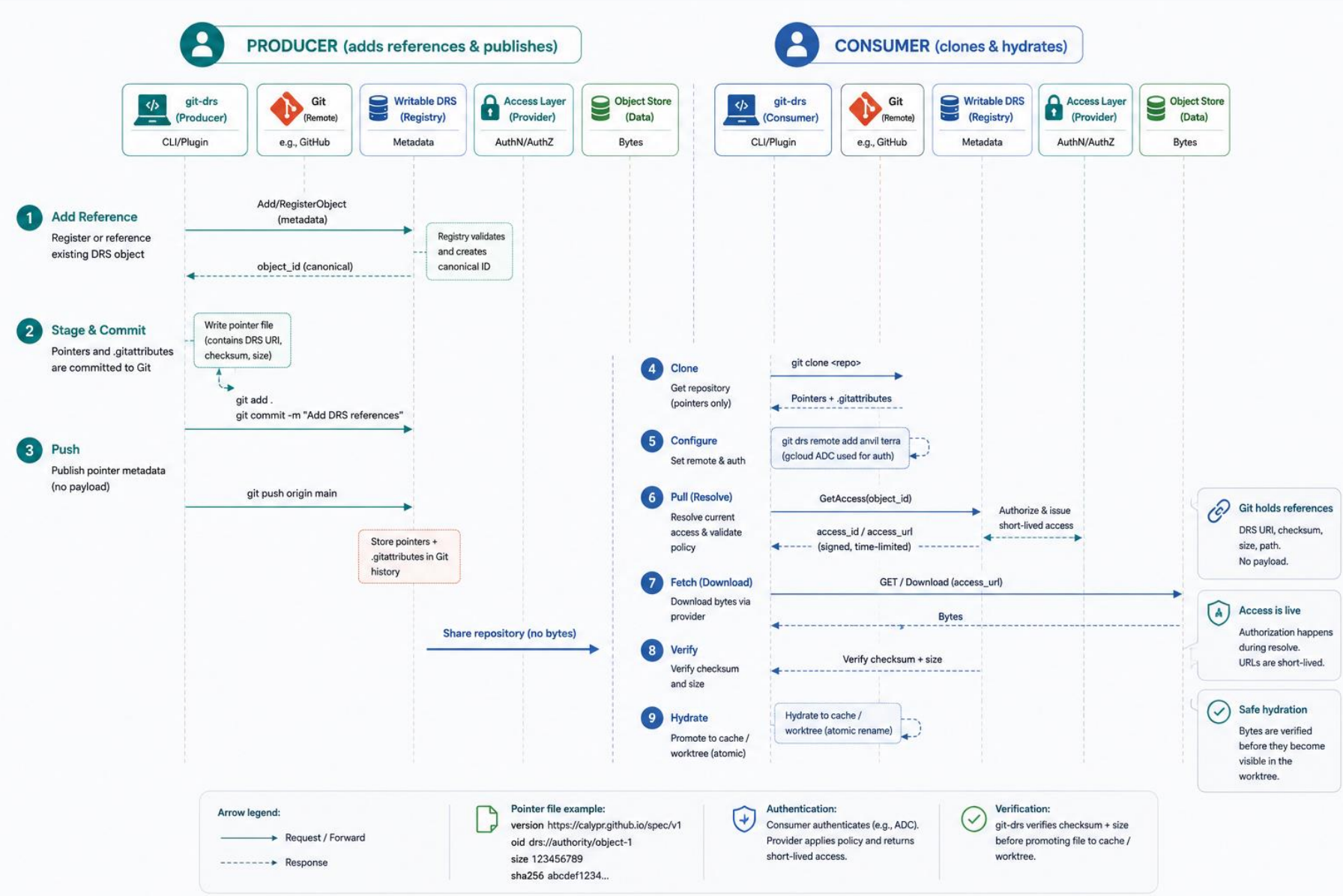
DRS references are stable and portable across platforms and time.



Fits your workflow

Use Git for versioning, review, automation, and releases.

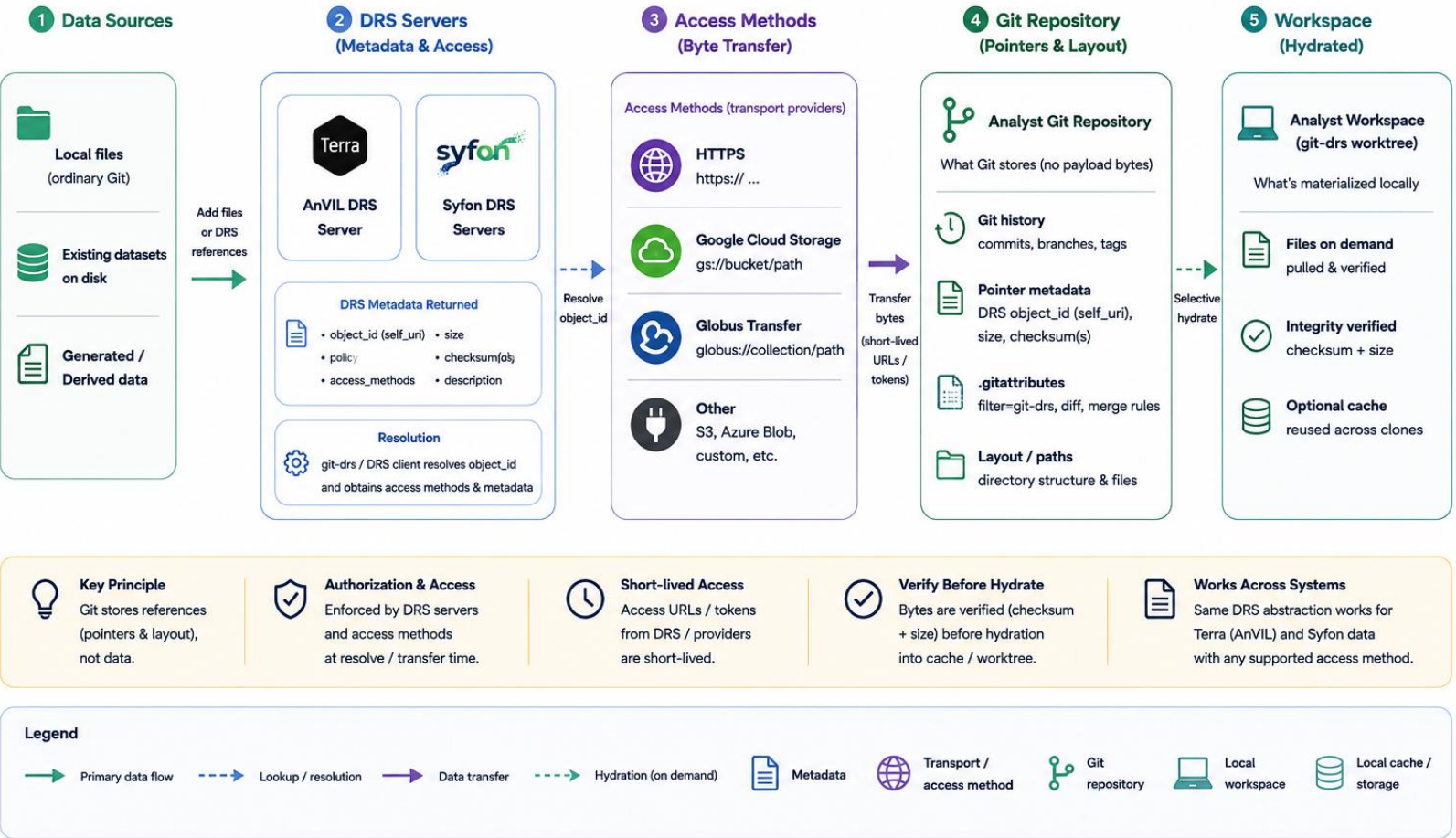
Writable DRS: register, update, and resolve



Analyst-facing Git repo with DRS-backed local, AnVIL, and Globus data

Git-DRS Architecture & Data Flow (Refined)

Git stores references (not data). DRS servers provide metadata & access. Access methods transfer bytes. git-drs hydrates files on demand.



Configure once. Work with ordinary Git.

```
# standard anvil credentials
gcloud auth application-default login

# Create and initialize git repository
mkdir my-anvil-repo
cd my-anvil-repo
git init
git drs init

# Configure the default drs remote
git drs remote add anvil terra
```

What this does

-  **1 Authenticate to AnVIL**
Uses your standard AnVIL credentials.
-  **2 Create repo**
Start a new Git repository and initialize git-drs.
-  **3 Add Terra (AnVIL) as the default DRS remote**
Resolve AnVIL DRS objects through the Terra provider.
-  **4 Ready to add data**
Add files/DRS references and publish with ordinary Git.



Key idea

Git versions stable DRS references; authorized bytes are resolved and hydrated only when needed.



https://github.com/EllrottLab/ANVIL_1000G_PRIMED_data_model

From AnVIL Data Explorer to versioned Git references

1 Select data in AnVIL

Use Data Explorer to find and select the files you need.

★ AnVIL Data Explorer

Table

Graph

Results: 1 – 7 of 7

	Name	File Format	Size	DRS URI
A	sequencing_dataset.tsv	tsv	12.4 MB	drs://drs.anv0:..abfe7
A	subject.tsv	tsv	78.2 kB	drs://drs.anv0:..4c14
A	sample_info.tsv	tsv	75.0 kB	drs://drs.anv0:..ed8f5
A	sample_qc.tsv	tsv	428.5 kB	drs://drs.anv0:..d6d7c
A	sequencing_metrics.tsv	tsv	0.24 kB	drs://drs.anv0:..9a0ca
A	sequencing_filter.tsv	tsv	1.32 MB	drs://drs.anv0:..0098e
A	subject.json	json	78.2 kB	drs://drs.anv0:..d2b9c

2 Add DRS references

Add the selected objects as DRS references in your repo.

```
# Add selected objects
git drs add-ref \
  drs://drs.anv0:v2_6d1cf5f0-
  99a8-3cd9-b580-01102f3abfe7 \
  sequencing_dataset.tsv

git drs add-ref \
  drs://drs.anv0:v2_b6b829f3-
  6046-3f29-896f-495a23f24c14 \
  subject.tsv

# Or add a manifest
git drs add-ref --manifest \
  my-references.tsv
```

3 Commit & publish

Commit pointer files and publish with ordinary Git.

```
git add .gitattributes '*.tsv'
```

```
git commit -m "Add AnVIL
dataset references"
```

```
git push
```



Your Git repository



Git contains the dataset layout and DRS references — not the AnVIL payloads.

Anyone with access can pull and hydrate the bytes when needed.

From Git references to hydrated AnVIL data

1 Clone Repository

Clone the pointer-only repository.
*.tsv files are DRS pointers,
not actual data.

Set up authentication

```
gcloud auth application-default login
```

Clone the pointer-only repository

```
git clone https://github.com/<user>/  
ANVIL_1000G_PRIMED_data_model.git  
cd ANVIL_1000G_PRIMED_data_model
```



After cloning, *.tsv files are DRS
pointers (not data).

2 Configure Terra Remote (Per Clone)

Configure a local Terra remote.
This step is required for every clone.

```
# Repository-local remote configuration;  
# not cloned from origin  
# This is the key security boundary:  
# every clone must recreate its own config  
git drs remote add anvil terra --checkout  
hydrate
```



This enables git-drs to:

- ✓ Resolve DRS URIs via the AnVILResolver
- ✓ Request fresh access tokens via Google ADC
- ✓ Verify file checksums during download

3 Hydrate Files

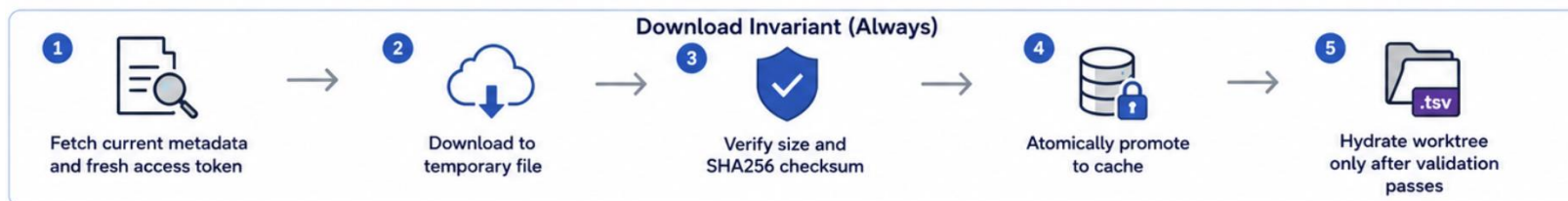
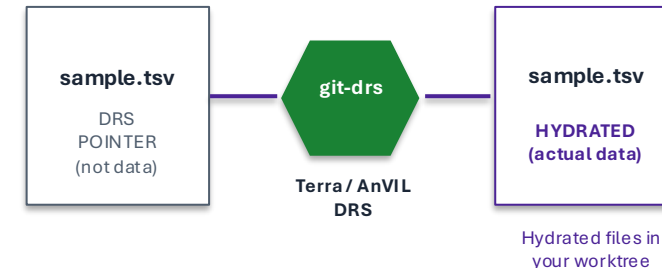
Download and hydrate files
from AnVIL (fetch metadata and data).

Download all authorized TSV files

```
git drs pull -I "*.tsv"
```

Or hydrate only a specific subset

```
git drs pull -I "sample.tsv"
```



Security & Integrity by Design:

Every clone recreates its own configuration and obtains fresh tokens. Data is downloaded securely, verified, and hydrated only after validation.